LLM Cost Tracker by PHANTSOM — Technical Documentation

How It Works (End-to-End) — Explained Simply

Author: Soham Dahivalkar (PHANTSOM)
Version: 1.0.1
Date: May 2026

What Is This?

This is a **VS Code Extension** — a small program that adds extra features to the VS Code editor. When a developer writes code that calls AI models (like ChatGPT, Claude, Gemini), this extension **automatically shows how much money each API call will cost** — right inside the code editor.

Think of it like a **price tag on every AI API call** in your code.

The Big Picture (How It Works in 5 Steps)

```
Step 1: You open a Python/JavaScript/TypeScript file in VS Code
            ↓
Step 2: Extension scans your code for AI model names (like "gpt-4o", "claude-
3")
            ↓
Step 3: It looks up the price of that model from its built-in price list
            ↓
Step 4: It calculates: "This call will cost $X per call, $Y per month"
            ↓
Step 5: It shows the cost RIGHT NEXT to your code — in color
```

That's it. No setup, no API keys, no internet needed. It just works.

How I Built It From Scratch

Tools I Used

Tool	What It Does	Why I Used It
TypeScript	Programming language (like JavaScript but with types)	VS Code extensions must be written in TypeScript/JavaScript
VS Code Extension API	Official toolkit from Microsoft to build extensions	This is how you add features to VS Code
Node.js	Runs JavaScript outside the browser	Needed to compile and package the extension
vsce	Command-line tool from Microsoft	Packages the extension into a .vsix file for publishing

Project Structure (What Each File Does)

```
llm-cost-estimator/
│
├── src/                        ← All source code lives here
│   ├── extension.ts            ← The "brain" — starts everything
│   ├── pricing.ts              ← Price list of 40+ AI models
│   ├── detector.ts             ← Finds AI calls in your code
│   ├── decorator.ts            ← Shows colored cost badges
│   ├── hoverProvider.ts        ← Shows detailed tooltip on mouse hover
│   ├── codeLensProvider.ts     ← Shows cost info ABOVE each line
│   └── statusBar.ts            ← Shows total cost at the bottom of VS Code
│
├── package.json                ← Extension config (name, commands, settings)
├── tsconfig.json               ← TypeScript compiler settings
├── README.md                   ← Marketplace description
└── resources/
    └── icon.png                ← Extension logo
```

Deep Dive: Each Module Explained

1. `extension.ts` — The Brain

What it does: This is the starting point. When VS Code loads, this file wakes up and connects all the other parts together.

How it works (simple version):

```
VS Code starts
    → extension.ts says "I'm alive!"
    → It creates the price calculator, the detector, the display system
    → It says "Hey VS Code, tell me whenever the user opens a file or types
something"
    → Every time the user types, it re-scans the code for AI calls
    → If it finds any, it shows the costs
```

Key things it handles:
- **Activation** — Starting up when VS Code opens a supported file
- **Event Listening** — Watching for file changes, tab switches, saves
- **Debouncing** — Waiting 400ms after you stop typing before scanning (so it doesn't slow down your typing)
- **Configuration** — Reading your settings (currency, budget, calls per day)
- **Budget Alerts** — Warning you if estimated costs exceed your budget

---

2. `pricing.ts` — The Price List

What it does: Contains the prices of 40+ AI models from 8 companies.

How it works (simple version):

```
It's like a menu at a restaurant:

  GPT-4o          → Input: $2.50 per million words, Output: $10.00 per
million words
  Claude 3 Haiku  → Input: $0.25 per million words, Output: $1.25 per million
words
  Gemini Flash    → Input: $0.10 per million words, Output: $0.40 per million
words
  ... and 37 more models
```

Each model entry stores:
- Model name (what developers type in their code)
- Display name (human-friendly name)
- Provider (OpenAI, Anthropic, Google, etc.)
- Input cost per 1 million tokens
- Output cost per 1 million tokens
- Context window size (how much text the model can read)
- Cost tier (cheap, moderate, expensive, premium)

How price lookup works:

```
Your code says: model="gpt-4o-2024-08-06"
    → Extension strips the date: "gpt-4o"
    → Looks up "gpt-4o" in the price list
    → Found! Input: $2.50/1M, Output: $10.00/1M
```

It handles tricky cases like:
- Date-suffixed models: `gpt-4o-2024-08-06` → matches `gpt-4o`
- Dash vs dot: `claude-3-5-sonnet` and `claude-3.5-sonnet` both work
- Case insensitive: `GPT-4O` and `gpt-4o` both match

Supported Providers (8 total):
1. OpenAI (GPT-4o, GPT-4, o1, o3, o4-mini, etc.)
2. Anthropic (Claude 3, 3.5, 4 — Opus, Sonnet, Haiku)
3. Google (Gemini 1.5, 2.0, 2.5 — Pro, Flash)
4. Mistral (Large, Small, Codestral, Nemo)
5. DeepSeek (V3, R1, Coder)
6. Cohere (Command R, Command R+)
7. Groq (LLaMA 3.1, Mixtral)
8. AWS Bedrock (Amazon Titan)

---

3. `detector.ts` — The Code Scanner

What it does: Reads your code line by line and finds every place where an AI model is being used.

How it works (simple version):

```
Your code:
    response = client.chat.completions.create(
        model="gpt-4o",        ← FOUND! Line 2, model = gpt-4o
        messages=[...],
        max_tokens=1000         ← FOUND! max_tokens = 1000
    )

Detector output:
    {
        line: 2,
        model: "gpt-4o",
        maxTokens: 1000,        ← extracted from nearby code
        callType: "chat"        ← it's a chat completion call
    }
```

What it searches for:
1. **Model name strings** — Uses pattern matching (regex) to find strings like `"gpt-4o"`, `"claude-3-sonnet"`, `"gemini-2.0-flash"` etc.
2. **max_tokens parameter** — Looks at lines near the model name (±10 lines) to find `max_tokens=1000` or similar
3. **Input token estimation** — Looks at nearby message/prompt strings and estimates how many tokens they contain (~4 characters = 1 token)
4. **Call type detection** — Figures out if it's a chat call, completion, embedding, etc.

Smart features:
- Skips comments (lines starting with `#` or `//`)
- Handles Python, JavaScript, and TypeScript syntax
- Works with OpenAI, Anthropic, Google, and all other provider code patterns

---

4. `decorator.ts` — The Visual Display

What it does: Takes the detected AI calls and shows colored cost labels right inside your code editor.

How it works (simple version):

```
Before (what you see normally):
    model="gpt-4o",

After (with extension active):
    model="gpt-4o",    ← 🟡 $0.003/call · ~$9.00/mo [GPT-4o]
```

Color coding system:

Color	Cost Per Call	Meaning
🟢 Green	Less than $0.001	Very cheap — go ahead!
🟡 Yellow	$0.001 to $0.01	Moderate — keep an eye on it
🟠 Orange	$0.01 to $0.05	Expensive — consider cheaper options
🔴 Red	$0.05 to $0.10	Very expensive — really think about this
💀 Skull	More than $0.10	Dangerously expensive!

3 display styles (user can choose):

1. **Badge** (default): `← 🟡 $0.003/call · ~$9.00/mo [GPT-4o]`
2. **Minimal**: `← $0.003/call`
3. **Detailed**: `← 🟡 $0.003/call · $0.30/day · $9.00/mo · OpenAI`

How the cost is calculated:

```
Input cost  = (estimated input tokens ÷ 1,000,000) × input price per million
Output cost = (max_tokens ÷ 1,000,000) × output price per million
Total cost  = Input cost + Output cost

Monthly cost = Total cost × calls per day × 30 days
```

---

5. `hoverProvider.ts` — The Detailed Tooltip

What it does: When you hover your mouse over a model name, it shows a rich popup with full pricing details.

What the popup shows:

```
┌──────────────────────────────────────┐
│  🟡 LLM Cost Estimator               │
│  ────────────────────                │
│  Model:          GPT-4o              │
│  Provider:       OpenAI              │
│  Context Window: 128,000 tokens      │
│                                      │
│  Pricing (per 1M tokens):            │
│  Input:   $2.50                      │
│  Output:  $10.00                     │
│  Cached:  $1.25                      │
│                                      │
│  This Call Estimate:                 │
│  Input Tokens:  ~500                 │
│  Output Tokens: ~1,000               │
│  Total Cost:    $0.003               │
│                                      │
│  Projections (100 calls/day):        │
│  Daily:   $0.30                      │
│  Weekly:  $2.10                      │
│  Monthly: $9.00                      │
│  Yearly:  $108.00                    │
└──────────────────────────────────────┘
```

---

6. `codeLensProvider.ts` — The Above-Line Info

What it does: Shows a clickable text ABOVE each API call line with cost summary.

What it looks like:

```
  🟡 GPT-4o — $0.003/call · $9.00/mo (100 calls/day)     ← This line is added by CodeLens
  💡 Consider gpt-4o-mini — 97% quality at 3% of the cost ← Cheaper alternative suggestion
      response = client.chat.completions.create(
          model="gpt-4o",
```

Smart suggestions: If you're using an expensive model, it suggests a cheaper alternative:
- Using GPT-4? → "Try gpt-4o-mini, 97% quality at 3% cost"
- Using Claude Opus? → "Try claude-3.5-haiku, 12x cheaper"

- Using Gemini Pro? → "Try gemini-2.0-flash for most tasks"

---

7. `statusBar.ts` — The Bottom Bar

What it does: Shows total file cost at the bottom of VS Code.

What it looks like:

```
┌──────────────────────────────────────────────────┐
│  [your code editor]                              │
│                                                  │
│                                                  │
├──────────────────────────────────────────────────┤
│  ⚡ LLM: 3 calls · ~$27.00/mo                    │  ← Status bar
└──────────────────────────────────────────────────┘
```

Budget alerts: If monthly cost exceeds your set budget, it shows:

```
│  ⚠️ LLM: 3 calls · ~$150.00/mo ⚠️ OVER BUDGET    │
```

How the Extension Lifecycle Works

```
1. INSTALL
   User installs from VS Code Marketplace
   Extension files are copied to VS Code extensions folder

2. ACTIVATE
   User opens a .py, .ts, or .js file
   VS Code calls our activate() function
   We set up all the components (detector, decorator, hover, codelens,
statusbar)

3. SCAN (happens automatically)
   User types code or opens a file
   We wait 400ms (debounce) to avoid scanning too often
   Quick check: does the file contain any AI model names?
   If no → do nothing (fast exit)
   If yes → full scan: find all models, extract max_tokens, estimate costs

4. DISPLAY
   Apply colored decorations to each line with a model name
   Update hover tooltips
   Update CodeLens above each call
   Update status bar with total cost
   Check budget alerts

5. REPEAT
   Every time the user edits, saves, or switches files → go back to step 3

6. DEACTIVATE
   User closes VS Code
   We clean up all decorations and status bar items
```

Configuration Options (What Users Can Customize)

Setting	What It Controls	Default
`llmCost.enabled`	Turn extension on/off	On
`llmCost.defaultInputTokens`	Assumed input tokens when not detectable	500
`llmCost.defaultOutputTokens`	Assumed output tokens when max_tokens not found	300
`llmCost.callsPerDay`	How many API calls per day (for monthly projections)	100
`llmCost.currency`	Display currency (USD, EUR, GBP, INR, JPY)	USD
`llmCost.monthlyBudget`	Budget alert threshold	0 (disabled)
`llmCost.showCodeLens`	Show/hide the above-line cost info	On
`llmCost.showStatusBar`	Show/hide bottom status bar	On
`llmCost.showHoverDetails`	Show/hide hover tooltips	On
`llmCost.decorationStyle`	Badge, minimal, or detailed style	Badge

Commands (What Users Can Run)

Press `Ctrl+Shift+P` in VS Code and type:

Command	What It Does
`LLM Cost: Toggle Inline Cost Display`	Turn cost badges on/off
`LLM Cost: Show File Cost Summary`	Show popup with full cost breakdown
`LLM Cost: Refresh Cost Estimates`	Re-scan the current file
`LLM Cost: Set Monthly Budget Alert`	Set a monthly spending limit

Supported Languages

Language	File Extensions
Python	`.py`
TypeScript	`.ts`
JavaScript	`.js`
TypeScript React	`.tsx`
JavaScript React	`.jsx`

How I Published It

Step 1: Build the code

```
TypeScript files (.ts) → compiled to → JavaScript files (.js)
```

Step 2: Package it

```
All files → packaged into → llm-cost-estimator-1.0.1.vsix
(A .vsix file is just a ZIP file with a specific structure)
```

Step 3: Upload to Marketplace

```
Upload .vsix to marketplace.visualstudio.com
Microsoft verifies it (5-10 minutes)
Extension goes LIVE — anyone can install it
```

Tech Stack Summary

Layer	Technology
Language	TypeScript
Runtime	Node.js
Platform	VS Code Extension API
Pattern Matching	Regular Expressions (Regex)
Build Tool	TypeScript Compiler (tsc)
Package Tool	vsce (VS Code Extension CLI)
Distribution	VS Code Marketplace
License	MIT (free for everyone)

What Makes This Extension Special

1. **Zero Configuration** — Install and it works. No API keys, no setup.
2. **Offline** — All pricing data is built-in. No internet needed.
3. **Real-time** — Updates as you type (with smart debouncing).
4. **40+ Models** — Covers all major AI providers.
5. **Multi-currency** — USD, EUR, GBP, INR, JPY.
6. **Budget Alerts** — Get warned before you overspend.
7. **Cheaper Alternatives** — Suggests cheaper models automatically.
8. **Lightweight** — Only 25 KB. Doesn't slow down VS Code.

Built by Soham Dahivalkar | PHANTSOM
"*The Invisible Force Behind Intelligent Systems*"